SDG Lens: Explainable Multi-Label Classification of UN Sustainable Development Goals

Abstract

We present SDG Lens, an attention-based multi-label classifier for identifying United Nations Sustainable Development Goals (SDGs) in UNDP's SDG Integration Corpus (SDGi) text. The system uses a partially fine-tuned BERT-family encoder (MiniLM) to predict one or more of the 17 SDG labels. To make the output easier to inspect, it also shows high-attention tokens for each prediction. We treat these tokens as proxy evidence, not as fully validated causal explanations. We compare the MiniLM model with a TF-IDF + linear SVM-style baseline using 4,000 training examples. The TF-IDF baseline reaches micro-F1 = 0.700, macro-F1 = 0.697, and subset accuracy = 0.586. MiniLM reaches micro-F1 = 0.717, macro-F1 = 0.706, and subset accuracy = 0.670, outperforming the baseline across all metrics. We also add a small manual inspection of top attention tokens on specific examples to check whether the model's highlights align with human expectations. The results show that SDG Lens is useful as a human-in-the-loop support tool, but it should not replace human judgement in policy or funding decisions.

1 Introduction

The United Nations Sustainable Development Goals (SDGs) represent a universal call to action, with 193 member states committed to addressing 17 interconnected goals spanning poverty, inequality, climate change, and institutional governance. Tracking progress toward these goals requires analyzing national reporting documents such as Voluntary National Reviews (VNRs) presented periodically at the UN High-level Political Forum. Manual SDG labelling can be slow and inconsistent. The same paragraph can relate to more than one SDG; for example, renewable energy text may belong to both SDG 7 and SDG 13. This makes the task multi-label classification.

Our research question is: can a compact BERT-family model classify VNR text into one or more SDG labels while also showing token-level evidence for human inspection?

In brief:

- The final MiniLM model trained on 4,000 examples reaches micro-F1 = 0.717 and macro-F1 = 0.706 on the held-out test set.
- The TF-IDF + linear SVM-style baseline reaches micro-F1 = 0.700 and macro-F1 = 0.697, showing that simple models can be very competitive in a low-data setting.
- Attention tokens often match meaningful SDG concepts, such as “infrastructure”, “housing”, “peace”, “greenhouse”, and “emissions”.
- Attention should be presented as inspectable proxy evidence, not as a fully validated explanation.

This report is structured as follows: Section 2 provides theoretical background on transformer architectures, multi-label learning, and attention mechanisms; Section 3 describes data collection, the SDGi corpus, and our model architecture; Section 4 presents results; Section 5 discusses implications, ethical considerations, and real-world use cases; Section 6 concludes.

2 Theory

Our approach builds on four theoretical foundations: transformer scaling laws, contextual semantic representation, multi-label learning theory, and the attention-as-explanation debate.

2.1 Transformer Scaling Laws and the Low-Data Challenge

TF-IDF models are strong baselines for text classification because they learn from word and phrase frequencies. Transformer models such as BERT and MiniLM usually need more data, but they can represent words in context. Research by Kaplan et al. (2020) demonstrates that model performance improves predictably with data and compute. Our expectation was that

MiniLM could get better as data grows.

2.2 Contextual Semantic Mapping

Traditional bag-of-words approaches like TF-IDF treat each word as an independent feature, losing syntactic structure and semantic context. BERT's contextual embeddings capture subtle distinctions: the word "infrastructure" carries different connotations when paired with "rural" versus "digital."

This capability is particularly important for SDGs, which exhibit significant semantic overlap. We would therefore expect BERT to outperform TF-IDF most clearly on SDGs with heavy semantic overlap, where word-level signals are ambiguous but contextual signals are strong.

2.3 Multi-Label Learning Theory

The SDGs are explicitly interconnected. The 2030 Agenda (UN General Assembly, 2015) defines the 17 goals as "integrated and indivisible," meaning that progress in one domain is structurally dependent on actions in others. This interlinkage has theoretical implications for multi-label classification:

- **Shared Vocabulary Effects:** Goals addressing similar themes share vocabulary. This can lead to both positive transfer (easier learning of related labels) and negative transfer (confusion between similar goals).
- **Label Co-occurrence Patterns:** In VNR reports, certain goals commonly appear together. Multi-label classifiers implicitly learn these correlations through shared classifier weights, whereas independent binary classifiers would duplicate effort.
- **Classification Threshold Dynamics:** With 17 labels, the probability of any single label being positive is low (average 1.44 labels per document). This sparsity explains why we use a threshold of 0.3 rather than 0.5.

Our multi-label architecture (sigmoid activation per label, independent thresholding) allows

the model to learn both synergies and distinctions between goals, unlike hierarchical approaches that predict goals sequentially. In practice, Vanderfeesten et al. (2021) have utilized a similar multi-label transformer approach to navigate these semantic overlaps in this specific SDG classification task. We would therefore expect multi-label architecture to help BERT outperform independent binary classifiers, the low threshold to be essential to avoid predicting zero labels, and shared encoder weights to enable positive transfer between semantically related SDGs.

2.4 Attention as Explanation

Our idea is inspired by explainable NLP work such as HateXplain (Mathew et al., 2021), where labels are connected to human rationales. Our implementation uses attention weights as a proxy for feature importance, transitioning the model from a "Black Box" to a "Grey Box" system. However, unlike HateXplain, the SDGi Corpus provides SDG labels but not gold token-level rationales. For this reason, we cannot claim that our attention tokens are human-validated explanations.

Moreover, this attention-as-explanation approach is theoretically contested. Jain & Wallace (2019) argue that attention weights do not always equal true feature importance. Wiegreffe & Pinter (2020) argue that attention can still be useful if we define clearly what it explains. We follow a cautious position: attention tokens help inspection, but they are not causal proof. Our report presents attention-weighted tokens alongside predictions and notes that without token-level rationale ground truth, these remain proxy explanations only.

3 Data and Methods

3.1 Data Collection and Annotation Process

We used the SDGi Corpus, a dataset of SDG-related text from Voluntary National Reviews and Voluntary Local Reviews. We used the English subset for this prototype. The raw test split contains 1,470 rows; after filtering to English, 1,057 rows remain and are used for final evaluation. Each text can have one or more of the 17 SDG labels.

It is important to describe our role clearly. We did not collect the original reports ourselves, and we did not create the official SDG labels from scratch. The SDG labels are provided by the SDGi Corpus. Our own contribution is the AI prototype, the baseline comparison, and a small manual rationale inspection of selected held-out examples.

Table 1: Corpus setup used in the current experiment.

Split / setting	Samples	Notes
Training set	4,000	Main training run for MiniLM and TF-IDF baseline
Smaller training runs	1,000 / 2,000	Used as an ablation to show low-data performance
Held-out test set	1,057	Used for final evaluation
Average labels per test document	1.44	Confirms this is a multi-label task

3.2 Preprocessing Pipeline

The text was preprocessed by converting text fields to string, removing null values, normalising whitespace, and filtering for English-language documents. We did not remove stopwords because transformer models can use function words as context.

3.3 Model Architecture

Our model uses a BERT-family encoder (sentence-transformers/all-MiniLM-L6-v2) with a modified classification head:

- **Encoder:** MiniLM (6 layers, 384 hidden dimensions), frozen except last 2 transformer layers
- **Pooling:** CLS token from last hidden state, where CLS is the transformer's special summary token
- **Dropout:** 10% hidden dropout
- **Classifier:** Linear layer mapping 384 $\rightarrow$ 17 (SDG labels)
- **Loss:** Binary Cross-Entropy with Logits (BCEWithLogitsLoss)

We use a low classification threshold (0.3) to prioritize recall. This ensures fewer relevant SDGs are missed, even at the cost of more false positives requiring human filtering. This represents a deliberate Human-AI Collaboration strategy: the AI acts as a discovery tool (finding all possible goals), and the human acts as the final filter (removing false positives).

The choice of sigmoid activation (versus softmax) is fundamental to multi-label learning. In multi-class problems (exactly one correct label), softmax produces a probability distribution summing to 1. In multi-label problems (any subset of labels can be correct), each label is independent. Sigmoid applies independently to each logit, computing $\sigma(\text{logit})$, which allows any combination of labels to be predicted. This is essential for SDGs where a document legitimately addresses multiple goals.

The baseline uses TF-IDF to convert each document into sparse word-importance features, then applies a linear SVM-style classifier to learn a separating boundary for each SDG label. In tables, we shorten this baseline name to "TF-IDF" for readability.

3.4 Implementation Details

Training parameters:

- Optimizer: AdamW with learning rate 5×10^{-5}
- Batch size: 8
- Epochs: 3
- Max sequence length: 128 tokens
- Device: CUDA (prototype)
- Seeds: 42, 43, 44, 45, 46

The model was implemented in PyTorch with HuggingFace Transformers, using the SDGi parquet data loaders and custom multi-label attention extraction.

3.5 Classification Threshold

Multi-label classification with 17 labels creates a sparse problem: on average only 1.44 labels are positive per document out of 17. This means a standard 0.5 threshold would predict zero labels for many documents. We select 0.3 as a recall-oriented threshold: for the BERT-4k artifacts, it produces an average of 1.42 predicted labels per document, close to the 1.44 average true labels, so that the AI acts as a discovery tool without usually returning empty predictions. This aligns with the multi-label formulation in Section 2.3, where a low threshold is essential to avoid predicting zero labels.

Table 2: Threshold sweep across 5 BERT-4k artifacts (1,057 test samples per seed). Asterisk marks selected threshold.

Threshold	Avg Predicted Labels	Avg True Labels	Micro-F1	Zero Predictions %
0.20	1.722	1.441	0.7021	3.1%
0.25	1.549	1.441	0.7136	4.7%
0.30	1.420	1.441	0.7206	6.3% *
0.35	1.321	1.441	0.7230	7.7%
0.40	1.237	1.441	0.7219	9.3%
0.45	1.161	1.441	0.7182	11.0%
0.50	1.093	1.441	0.7127	12.7%

3.6 Attention-Based Evidence and Manual Rationale Check

After prediction, we extract high-attention tokens and display them next to the predicted labels. These tokens are not treated as final explanations. They are shown as evidence that a human can inspect.

Concretely, we take the final transformer layer's attention from the `[CLS]` token to each input token, average it across attention heads, mask padding and special tokens, and sort the remaining tokens by score. Therefore, "top attended" means highest average `[CLS]`-attention tokens, not words proven to cause the prediction.

To connect this more clearly to HateXplain, we add a small manual rationale check. For five held-out examples, we manually inspect whether the top attention tokens match the words that a human would expect for the SDG label. This is qualitative and small, but it makes the

explainability claim more careful and transparent.

3.7 Evaluation Metrics

We report micro-F1, macro-F1, weighted-F1, subset accuracy, and per-SDG F1. Micro-F1 measures overall label performance, macro-F1 treats each SDG equally, and subset accuracy is stricter because the full predicted label set must match the gold label set.

4 Results

4.1 Overall Performance: Comparison with TF-IDF Baseline

Table 3: Mean $\pm$ standard deviation over seeds 42, 43, 44, 45, and 46.

Model	Train Size	Micro-F1	Macro-F1	Subset Acc.	Train Time
BERT	1,000	0.657 ± 0.011	0.654 ± 0.011	0.635 ± 0.034	11.5 ± 0.35
BERT	2,000	0.687 ± 0.010	0.682 ± 0.009	0.650 ± 0.024	22.0 ± 0.17
BERT	4,000	0.717 ± 0.003	0.706 ± 0.002	0.670 ± 0.013	44.6 ± 0.60
TF-IDF	1,000	0.607 ± 0.013	0.605 ± 0.012	0.459 ± 0.017	1.8 ± 0.06
TF-IDF	2,000	0.675 ± 0.011	0.669 ± 0.012	0.562 ± 0.020	2.9 ± 0.16
TF-IDF	4,000	0.700 ± 0.005	0.697 ± 0.008	0.586 ± 0.005	5.0 ± 0.12

As shown in Table 3, BERT outperforms TF-IDF at every training size we tested: 0.657 vs 0.607 at 1,000 samples, 0.687 vs 0.675 at 2,000 samples, and 0.717 vs 0.700 at 4,000 samples. This runs counter to the expectation in Section 2.1 that compact transformer models underperform simple baselines in low-data regimes—BERT's contextual embeddings give it an advantage even at 1,000 examples. Both models improve with scale, but BERT's advantage is consistent throughout.

4.2 Per-Label Performance

Performance varies significantly across SDGs:

- **Best performers:** SDG 11 (Sustainable Cities, F1=0.764), SDG 9 (Industry, Innovation, F1=0.752), SDG 13 (Climate Action, F1=0.749), SDG 4 (Quality Education, F1=0.741)

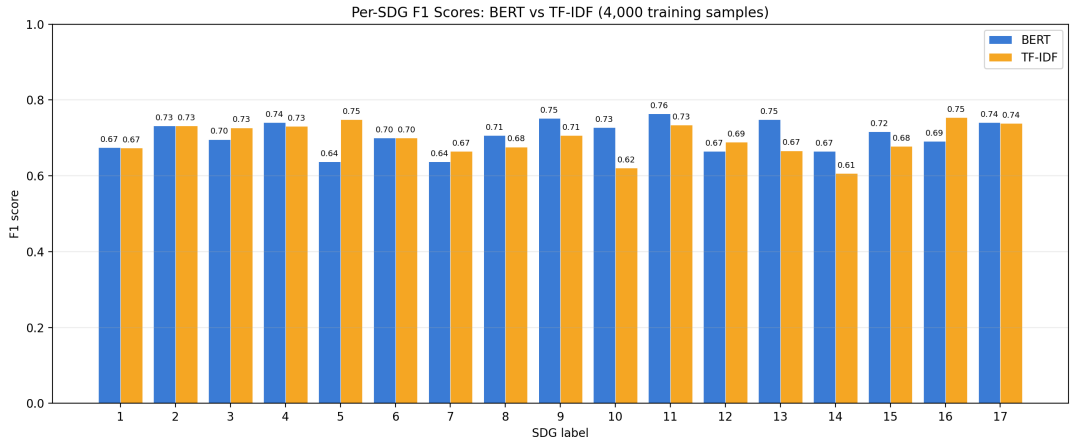

Figure 1: Per-SDG F1 scores comparing BERT and TF-IDF at 4,000 training samples. Blue bars show BERT, orange bars show TF-IDF.

- **Weakest performers:** SDG 5 (Gender Equality, F1=0.637), SDG 7 (Clean Energy, F1=0.637), SDG 14 (Life Below Water, F1=0.665)

The variation reflects both label frequency and semantic ambiguity in the VNR documents. SDG 11 (Sustainable Cities) and SDG 9 (Industry, Innovation) benefit from concrete vocabulary ("infrastructure," "urban," "industry") that creates a consistent feature space. SDG 5 (Gender Equality) and SDG 1 (No Poverty) use more abstract or politically sensitive language that varies across countries, leading to poorer performance.

The per-label pattern partly supports the contextual-overlap expectation in Section 2.2, but not uniformly. At 4,000 training samples, BERT's largest gains are on SDG 10 (+0.106), SDG 13 (+0.082), SDG 14 (+0.059), SDG 9 (+0.046), and SDG 11 (+0.030), while SDG 6 is almost tied (+0.001). TF-IDF still leads on SDG 5 (+0.111), SDG 16 (+0.063), SDG 3 (+0.031), SDG 7 (+0.028), and SDG 12 (+0.024), showing that sparse lexical features remain strong for several goals.

4.3 Attention-Based Token Examples

Figure 2 demonstrates how attention-weighted tokens help the human analyst: instead of reading all 128 tokens, they can glance at top tokens like "housing" or "urban" for SDG 11 to quickly verify the AI's logic. For example, on an ocean conservation text, BERT highlights "oceans," "marine," "seas," and "conserve", which is exactly the vocabulary a human domain

expert would expect for SDG 14 (Life Below Water). On a flood mitigation text, it attends to "floods," "flooding," and "plan," appropriately capturing water management, climate action, and institutional language.

These tokens are proxy evidence only. Without gold token-level rationales, we cannot claim they are causally validated explanations.

5 Discussion of Results

5.1 Data Representation Limitations

Our prototype uses English-only documents, introducing potential linguistic and cultural bias. Non-Western contexts may interpret SDGs differently, and the model's vocabulary learned primarily from English-language VNRs may not transfer to other languages.

Furthermore, VNRs represent official government narratives, which may emphasize progress while underreporting challenges. This systematic bias in source documents affects model training. Countries may strategically frame their reports to highlight achievements, creating a positive bias in the training data.

This limitation has ethical implications: the model learns from politically filtered information and may reproduce these biases in downstream applications.

5.2 Real-World Use Cases

We propose two use cases for SDG Lens: an analyst reviewing VNR reports uses the model as a recall-oriented pre-screener (threshold 0.3) and filters false positives by checking attention tokens; an NGO compares predictions against a country's official project database to flag discrepancies.

We recommend that all automated SDG classifications be reviewed by human analysts before official use.

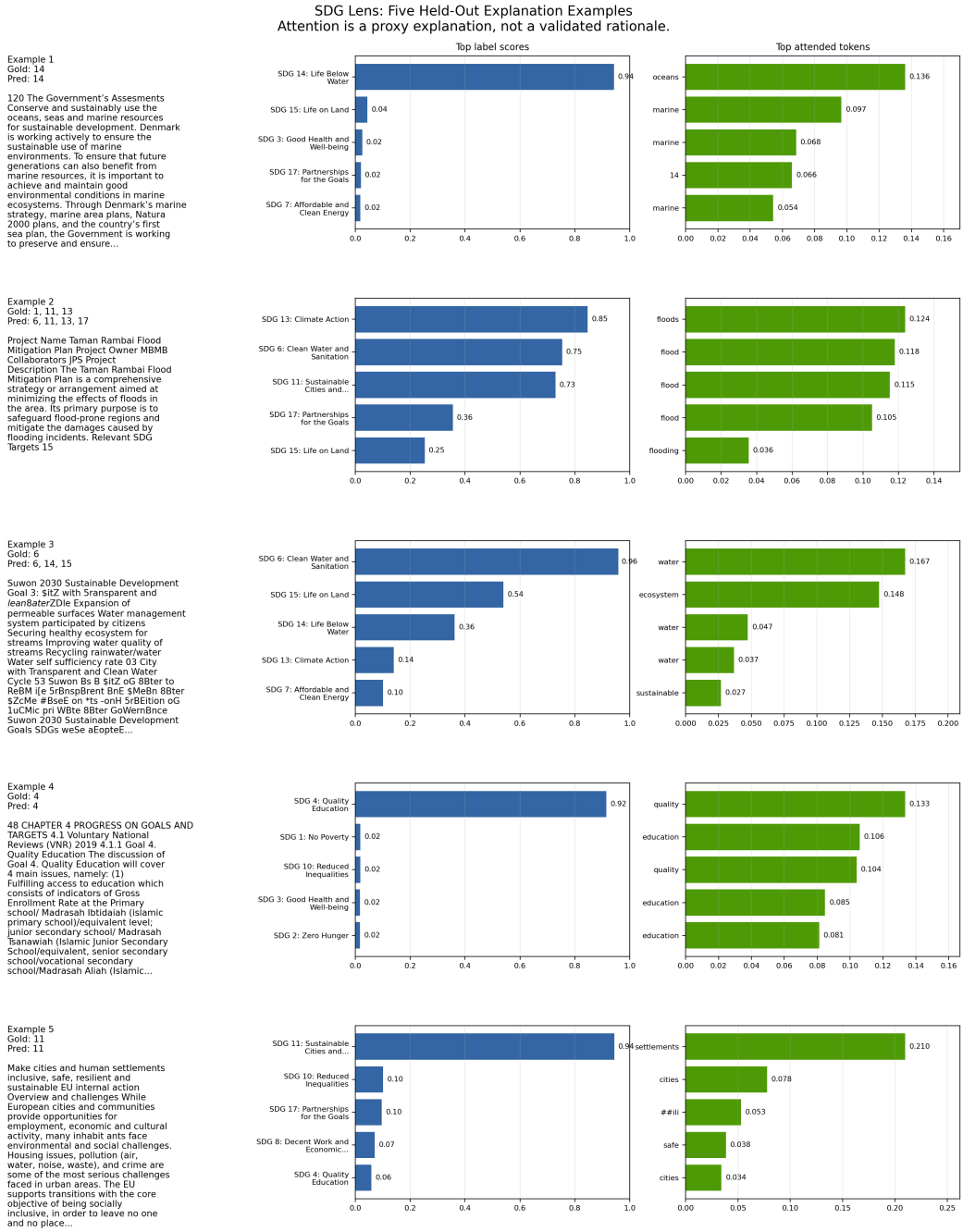

Figure 2: Example attention explanations showing top-attended tokens that help human analysts quickly verify predictions. Attention is a proxy explanation, not a validated rationale.

6 Conclusion

This project demonstrates the feasibility of explainable multi-label classification for UN Sustainable Development Goals. Key findings:

- BERT-based classifiers achieve micro-F1=0.717 at 4,000 samples (vs TF-IDF's 0.700), with significant per-label variance (SDG 11: 0.764 vs SDG 5: 0.637); future work should explore class-balancing techniques, larger encoders, and multi-language VNRs
- Attention-weighted tokens provide plausible rationales for human verification; they should be treated as inspectable proxy evidence, not causal explanations

The SDG Lens serves as a prototype for building trust with non-technical stakeholders (NGOs, UN researchers) by providing inspectable evidence for every prediction. This interpretability is not merely a technical requirement but an ethical mandate for high-stakes policy applications.

References

[1] Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Childs, R., ... & Amodei, D. (2020). Scaling laws for neural language models. *arXiv:2001.08361*. https://doi.org/10.48550/arXiv.2001.08361

[2] UN General Assembly. (2015). Transforming our world: the 2030 Agenda for Sustainable Development (A/RES/70/1). https://undocs.org/A/RES/70/1

[3] Mathew, B., Saha, P., Yimam, S. M., Biemann, C., Goyal, P., & Mukherjee, A. (2021). HateXplain: A benchmark dataset for explainable hate speech detection. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(17), 14867–14875. https://doi.org/10.48550/arXiv.2012.10289

[4] Vanderfeesten, M., Otten, R., & Schmidt, F. (2021). AI for mapping multi-lingual academic papers to the United Nations' Sustainable Development Goals (SDGs). *Zenodo*. https://doi.org/10.5281/zenodo.5603019

[5] Jain, S., & Wallace, B. C. (2019). Attention is not Explanation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 3543–3556. https://doi.org/10.18653/v1/D19-1357

[6] Wiegreffe, S., & Pinter, Y. (2020). Attention is not not Explanation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 11–20. https://doi.org/10.18653/v1/D19-1002

[7] AI for Global Challenges. (2026). SDG Lens: Data and Code. https://github.com/qmanhbeo/AI4GC-group

A Reproducibility Note

This appendix describes the repository workflow exactly as implemented. The marker-facing command is `pythonmain.pysweep`. It runs five stages in order: BERT training, TF-IDF baseline training, artifact evaluation, visualization generation, and LaTeX compilation. By default, the sweep uses training sizes `1000`, `2000`, and `4000`; seeds `42`, `43`, `44`, `45`, and `46`; language `en`; test seed `43`; threshold `0.3`; and device `cuda`. If all artifact metadata and referenced model files already exist, the training stages skip retraining unless `--force` is passed.

A.1 Data Source and Files

The local data files are copied SDGi Corpus parquet files stored as `data/train-00000-of-00001.parquet` and `data/test-00000-of-00001.parquet`. The dataset is the UNDP SDG Integration Corpus (SDGi Corpus), described in the repository `https://github.com/UNDP-Data/dsc-sdgi-corpus` and paper `https://ceur-ws.org/Vol-3764/paper3.pdf`. The source texts come from Voluntary National Reviews (`https://hlpf.un.org/vnrs`) and Voluntary Local Reviews (`https://sdgs.un.org/topics/voluntary-local-reviews`); the local dataset card lists license `CCBY-NC-SA4.0`. The raw split contains `5880` train rows and `1470` test rows. After filtering to English, the current repository has `4225` train rows and `1057` test rows. Each row contains `text`, `embedding`, `labels`, and `metadata`; the model pipeline

uses text, labels, and metadata. Labels are 1-based SDG identifiers from 1 to 17.

A.2 Sampling and Preprocessing

Both model families filter by metadata["language"]=="en" and use the same coverage-aware sampling rule. If the requested sample size is less than the filtered split, the sampler first selects at least one available row for each SDG using random.Random(seed), fills the remaining slots from a seeded shuffle of all other rows, truncates to the requested size, and shuffles the selected rows again. If the requested size is non-positive or at least the split size, it uses the full split after pandas.sample(frac=1.0,random_state=seed). For the test set, the sweep passes test_seed=43; because 1470 exceeds the English test size, all 1057 English test rows are used in shuffled order.

TF-IDF text preprocessing replaces standalone numbers with NUM, removes punctuation, collapses whitespace, lowercases, and strips leading/trailing whitespace. BERT text preprocessing only converts missing text to an empty string, replaces the Unicode non-character U+FFFE with a space, collapses whitespace, and strips.

A.3 BERT Training

The BERT-family model is sentence-transformers/all-MiniLM-L6-v2. The tokenizer is loaded with use_fast=True; by default the code uses local HuggingFace files only unless --allow-download is passed. Tokenization uses truncation, dynamic padding, return_tensors="pt", and max_length=128. The model loads AutoConfig with output_attentions=True and eager attention where supported, then loads AutoModel. The encoder is frozen except for the final 2 transformer layers by default; passing --unfreeze-encoder trains all encoder layers. The classifier pools the final hidden state of the [CLS] token, applies dropout using the encoder hidden dropout probability or 0.1, and maps the hidden size to 17 logits with one linear layer. The loss is BCEWithLogitsLoss.

For each BERT run, the script seeds Python, NumPy, and PyTorch with the run seed. Training uses AdamW, learning rate 5e-5, batch size 8, 3 epochs, no scheduler, no class weighting, and

gradient clipping at norm 1.0. The training loader shuffles batches; the test loader does not. Probabilities are sigmoid(logits). Predictions use probability>0.3; if a row has no positive labels after thresholding, the highest-probability SDG is forced on as a top-1 fallback. Each run saves a checkpoint model.pt, run_config.json, metrics.json, examples.jso n, results.json, and an artifact.json handoff file under artifacts/bert_train\{si ze\}_seed\{seed\}/.

A.4 TF-IDF Baseline

The baseline converts labels with MultiLabelBinarizer(classes=range(1,18)). It uses TfidfVectorizer(strip_accents="ascii",lowercase=False,ngram_range=(1,1) ,min_df=10,max_df=0.9,max_features=10000,binary=True). It trains one SGDCla ssifier per SDG label with loss="hinge", penalty="l2", alpha=1e-4, shuffle=True, learning_rate="optimal", random_state=seed, early_stopping=True, valida tion_fraction=0.2, and class_weight="balanced". If a label column has only one class in a training slice, the saved model list stores None for that SDG. During baseline training metrics, such a column predicts the constant training value; during the separate evaluation stage, None columns predict 0. Each baseline artifact saves results.json, tfidf_model.j oblib, and artifact.json under artifacts/tfidf_train\{size\}_seed\{seed\}/.

A.5 Evaluation, Sweeps, and Reporting

The evaluation stage discovers all artifact.json files under artifacts/. BERT artifacts are reloaded from model.pt, rebuilt with the checkpoint model name, threshold, batch size, max length, trainable-layer setting, and language, then evaluated on the same English test split. TF-IDF artifacts reload the saved vectorizer, per-label classifiers, and binarizer, then recompute predictions on the same English test split. Metrics are micro-F1, macro-F1, weighted-F1, subset accuracy, and per-label F1 with zero_division=0. The summary groups rows by model_t ype and train_size, then reports means and sample standard deviations across available seeds.

The evaluation stage writes `results/evaluation\_by\_seed.csv`, `results/evaluation\_summary.csv`, `results/evaluation\_summary.json`, and `results/evaluation\_summary.md`. It also writes two threshold sweeps at thresholds `0.20`, `0.25`, `0.30`, `0.35`, `0.40`, `0.45`, and `0.50`: `threshold\_sweep.json` pools all evaluated artifacts, while `threshold\_sweep\_bert4k.json` pools only BERT artifacts with `train\_size=4000`. These sweeps use `probability>=threshold` and do not apply the BERT top-1 empty-prediction fallback. TF-IDF sweep scores are produced by applying a sigmoid transform to each classifier's `decision\_function` margin; these are diagnostic scores, not calibrated probabilities.

The visualization stage reads the evaluation CSV/JSON files. It writes summary tables to `outputs/tables/`, plots micro-F1 and macro-F1 bar charts, plots BERT-vs-TF-IDF per-label F1 at 4000 training samples, writes the manuscript threshold table from `threshold\_sweep\_bert4k.json`, and selects the largest seed-42 BERT result file for single-artifact metric, per-label, and explanation-example figures. Explanation figures show at most five examples. For each example, the report displays top label probabilities and the five highest-scoring non-special, non-padding attention tokens. Generated tables and charts are copied into `manuscript/visualization/`. The manuscript compiler checks that the required table and comparison charts exist, runs `pdflatex` twice with `-interaction=nonstopmode-halt-on-error`, writes the PDF, and removes temporary LaTeX files.